

Отчет по Лабораторной работе №2:

Оптимизация и программирование БД

1. Наполнение базы данных

Для тестирования системы была использована автоматическая генерация данных.

- **Инструмент:** Python-скрипт (`lab2/generate_data.py`) с использованием библиотеки `uuid` и `random` .
- **Объем данных:** В каждую основную таблицу было добавлено 60+ записей (всего более 2500 записей в логах).
- **Метод загрузки:** CSV-файлы → команда `COPY` через Docker-контейнер PostgreSQL.

Результаты загрузки:

- `users` : 125 записей
- `patients` : 60 записей
- `doctors` : 60 записей
- `prescriptions` : 120 записей
- `intake_logs` : 1464 записи

2. Реляционные SQL-запросы

Были реализованы 5 сложных запросов для решения функциональных требований системы:

1. **Поиск пациентов с высокой нагрузкой (3+ назначения):** Используются `JOIN` , `GROUP BY` , `HAVING` .
2. **Статистика по специализациям врачей:** `JOIN` врачей и назначений, агрегация по специализации.
3. **Поиск соответствия "Диагноз → Препарат":** Многосторонний `JOIN` 6 таблиц с фильтрацией по названиям.
4. **Формирование ежедневного графика:** `JOIN` всех сущностей с фильтрацией по текущей дате.
5. **Анализ приверженности (процент пропусков):** Агрегатные функции, `CASE WHEN` , расчет процентов.

3. Анализ производительности и Оптимизация

Выбранные запросы для анализа:

Запрос 3 (Поиск соответствия диагноза и препарата):

```
SELECT p.full_name, dd.name as diagnosis, dr.trade_name as drug
FROM patients p
```

```
JOIN patient_diagnoses pd ON p.patient_id = pd.patient_id
JOIN diagnoses_directory dd ON pd.diag_id = dd.diag_id
JOIN prescriptions pr ON p.patient_id = pr.patient_id
JOIN prescription_items pi ON pr.prescription_id = pi.prescription_id
JOIN drugs_directory dr ON pi.drug_id = dr.drug_id
WHERE dd.name = 'Гипертония' AND dr.trade_name = 'Эналаприл';
```

Запрос 4 (Формирование ежедневного графика приемов):

```
SELECT p.full_name, dr.trade_name, s.time_of_day, pi.dosage
FROM patients p
JOIN prescriptions pr ON p.patient_id = pr.patient_id
JOIN prescription_items pi ON pr.prescription_id = pi.prescription_id
JOIN drugs_directory dr ON pi.drug_id = dr.drug_id
JOIN intake_schedules s ON pi.item_id = s.item_id
WHERE pr.start_date <= CURRENT_DATE AND (pr.end_date IS NULL OR pr.end_date >= CURRENT_DATE)
ORDER BY s.time_of_day;
```

Запрос 5 (Анализ приверженности лечению / % пропусков):

```
SELECT p.full_name,
       ROUND(COUNT(CASE WHEN il.status = 'Skipped' THEN 1 END) * 100.0 / COUNT(*), 2) as skip_percentage
FROM patients p
JOIN prescriptions pr ON p.patient_id = pr.patient_id
JOIN prescription_items pi ON pr.prescription_id = pi.prescription_id
JOIN intake_schedules s ON pi.item_id = s.item_id
JOIN intake_logs il ON s.schedule_id = il.schedule_id
GROUP BY p.patient_id, p.full_name
HAVING COUNT(*) > 0
ORDER BY skip_percentage DESC;
```

Результаты EXPLAIN ANALYZE:

Запрос	Время (До)	Время (После)	Изменение в плане
Запрос 3	0.385 ms	0.360 ms	Появление Index Scan по dr.trade_name и dd.name
Запрос 4	0.222 ms	0.220 ms	Оптимизация фильтрации дат через idx_prescriptions_dates
Запрос 5	1.804 ms	1.572 ms	Ускорение доступа к логам через idx_intake_logs_status

Визуальные планы выполнения и анализ:

Запрос 3 (Соответствие диагноза и препарата):

- | # | node, ms | tree, ms | rows | RRbF | Q | Plan | sh.ht | sh.dr |
|----|----------|----------|------|-------|----|--|-------|-------|
| | .210 | 20 | 8 | | | итоговые результаты (21KB = rows=20 x width=1'079) | 8 | |
| 0 | .018 | .210 | 20 | | | Hash Join | | |
| 1 | .019 | .169 | 20 | | | -> Hash Join | | |
| 2 | .032 | .060 | 53 | | | -> Hash Join | | |
| 3 | .013 | | 240 | | HR | -> Seq Scan on prescription_items pi | 2 | |
| 4 | .011 | .015 | 1 | | | -> Hash | | |
| 5 | .004 | 1 | 4 | 80.0% | | -> Seq Scan on drugs_directory dr | 1 | |
| 6 | .011 | .090 | 43 | | | -> Hash | | |
| 7 | .023 | .079 | 43 | | | -> Hash Join | | |
| 8 | .011 | | 120 | | | -> Seq Scan on prescriptions pr | 2 | |
| 9 | .007 | .045 | 22 | | | -> Hash | | |
| 10 | .024 | .038 | 22 | | | -> Hash Join | | |
| 11 | .007 | | 120 | | HR | -> Seq Scan on patient_diagnoses pd | 1 | |
| 12 | .005 | .007 | 1 | | | -> Hash | | |
| 13 | .002 | 1 | 4 | 80.0% | | -> Seq Scan on diagnoses_directory dd | 1 | |
| 14 | .014 | .023 | 60 | | | -> Hash | | |
| 15 | .009 | | 60 | | | -> Seq Scan on patients p | 1 | |
| | | | | | | Planning | 446 | 1 |
| | 2.951 | | | | EP | Planning Time | | |
| | .126 | .336 | | | | Execution Time | | |

- | # | node, ms | tree, ms | rows | RRBF | Q | Hash Join | Hash J | Hash Join | Seq Scs | Seq | Hash Join | Seq Scar | Hash Jo | Seq | Seq | Seq Sc | sh.ht | |
|----|----------|----------|------|---------|----|--|--------|-----------|---------|-----|-----------|----------|---------|-----|-----|--------|-------|----|
| | .227 | 20 | 8 | | | итоговые результаты (21KB = rows=20 x width=1'079) | | | | | | | | | | | 8 | |
| 0 | .019 | .227 | 20 | ▲ | | Hash Join | | | | | | | | | | | | |
| 1 | .013 | .179 | 20 | ▲ | | -> Hash Join | | | | | | | | | | | | |
| 2 | .032 | .062 | 53 | ▲ | | -> Hash Join | | | | | | | | | | | | |
| 3 | .014 | | 240 | | HR | -> Seq Scan on prescription_items pi | | | | | | | | | | | | 2 |
| 4 | .009 | .016 | 1 | | | -> Hash | | | | | | | | | | | | |
| 5 | .007 | | 1 | 4 80.0% | | -> Seq Scan on drugs_directory dr | | | | | | | | | | | | 1 |
| 6 | .011 | .104 | 43 | | | -> Hash | | | | | | | | | | | | |
| 7 | .021 | .093 | 43 | | | -> Hash Join | | | | | | | | | | | | |
| 8 | .016 | | 120 | | | -> Seq Scan on prescriptions pr | | | | | | | | | | | | 2 |
| 9 | .008 | .056 | 22 | ▼ | | -> Hash | | | | | | | | | | | | |
| 10 | .015 | .048 | 22 | ▼ | | -> Hash Join | | | | | | | | | | | | |
| 11 | .008 | | 120 | | HR | -> Seq Scan on patient_diagnoses pd | | | | | | | | | | | | |
| 12 | .022 | .025 | 1 | | | -> Hash | | | | | | | | | | | | |
| 13 | .003 | | 1 | 4 80.0% | | -> Seq Scan on diagnoses_directory dd | | | | | | | | | | | | 1 |
| 14 | .016 | .029 | 60 | | | -> Hash | | | | | | | | | | | | |
| 15 | .013 | | 60 | | | -> Seq Scan on patients p | | | | | | | | | | | | 1 |
| | | | | | | Planning | | | | | | | | | | | | 16 |
| | 1.049 | | | | EP | Planning Time | | | | | | | | | | | | |
| | .133 | .360 | | | | Execution Time | | | | | | | | | | | | |

- | # | node, ms | tree, ms | rows | RRBF | Q | loops | ... | Hash Join | Seq Sc | Hash Jo | Seq Hash Join | Seq Scan on pri | sh.ht |
|----|----------|----------|------|------|--------|-------|--|-----------|--------|---------|---------------|-----------------|-------|
| | | .138 | | 120 | | | ▼ итоги результаты | | | | | | 10 |
| 0 | .060 | .138 | | | | | Sort | | | | | | 5 |
| 1 | .023 | .078 | | | | | -> Hash Join | | | | | | |
| 2 | .008 | | 1 ▼ | | | | -> Seq Scan on intake_schedules s | | | | | | 1 |
| 3 | | .047 | | | | | -> Hash | | | | | | |
| 4 | .001 | .047 | | | | | -> Nested Loop | | | | | | |
| 5 | .010 | .046 | | | | | -> Hash Join | | | | | | |
| 6 | .003 | | 1 ▼ | | | | -> Seq Scan on patients p | | | | | | 1 |
| 7 | | .033 | | | | | -> Hash | | | | | | |
| 8 | .012 | .033 | | | | | -> Hash Join | | | | | | |
| 9 | .003 | | 1 ▼ | | | | -> Seq Scan on prescription_items pi | | | | | | 1 |
| 10 | | .018 | | | | | -> Hash | | | | | | |
| 11 | .018 | | | 120 | 100.0% | SR | -> Seq Scan on prescriptions pr | | | | | | 2 |
| 12 | | | n/e | | | | -> Index Scan using drugs_directory_pkey on drugs_directory dr | | | | | | |
| | | | | | | | Planning | | | | | | 86 |
| | 1.054 | | | | | EP | Planning Time | | | | | | |
| | .076 | .214 | | | | | Execution Time | | | | | | |

- SR #11 EP Planning Time

#0 S | #2 SS | #11 SS | tilemap | piechart

#	node, ms	tree, ms	rows	RRBF	Q	loops	Sort	Hash	Seq Scan on intake_schedules s	Hash	Hash	Seq Sc	sh.ht	sh.ud
		.268		120			▼ итоги						scan=10	9 1
0	.048	.268					Sort							5
1	.011	.220					-> Hash Join							
2	.159		1 ▼				-> Seq Scan on intake_schedules s							1
3	.050						-> Hash							
4	.050						-> Nested Loop							
5	.011	.050					-> Hash Join							
6	.003		1 ▼				-> Seq Scan on patients p							1
7	.036						-> Hash							
8	.016	.036					-> Hash Join							
9	.003		1 ▼				-> Seq Scan on prescription_items pi							1
10	.017						-> Hash							
11	.017			120 100.0%										2
12			n/e				-> Index Scan using drugs_directory_pkey on drugs_directory dr							
							Planning							87 3
	1.089						EP Planning Time							
	.088	.356					Execution Time							

- План до оптимизации: ![Запрос 5 До](png for lab2/запрос 5 до индекса.jpg)

- План после оптимизации:

#	node_ms	tree_ms	rows	Sort	HashAggregate	Hash Join	Hash Join	Hash Join	Hash Join	Hash Join	Seq Sc	Hash	Seq	Hash	Seq	Hash	Seq	sh.ht
	1.194	53		итоговые результаты (4.9KB = rows=53 x width=95)														23
0	.046	1.194	53	▲ Sort														3
1	.209	1.148	53	▲	-> HashAggregate													
2	.167	.939	1'464		-> Hash Join													
3	.158	.749	1'464		-> Hash Join													
4	.157	.559	1'464		-> Hash Join													
5	.213	.350	1'464		-> Hash Join													
6	.057		1'464		-> Seq Scan on intake_logs il													11
7	.048	.080	481		-> Hash													
				развернуть/свернуть узел ▶	-> Seq Scan on intake_schedules s													4
9	.031	.052	240		-> Hash													
10	.021		240		-> Seq Scan on prescription_items pi													2
11	.023	.032	120		-> Hash													
12	.009		120		-> Seq Scan on prescriptions pr													2
13	.015	.023	60		-> Hash													
14	.008		60		-> Seq Scan on patients p													1
				Planning														108
	.935			Planning Time														
	.107	1.301		Execution Time														

Применяемые индексы:

```
-- Оптимизация Запроса 3: Поиск по названиям диагноза и препарата
CREATE INDEX idx_diag_name ON diagnoses_directory(name);
CREATE INDEX idx_drug_trade_name ON drugs_directory(trade_name);

-- Оптимизация Запроса 4: Фильтрация по датам назначений
CREATE INDEX idx_prescriptions_dates ON prescriptions(start_date, end_date);

-- Оптимизация Запроса 5: Фильтрация и группировка по статусам логов и связям
CREATE INDEX idx_intake_logs_status ON intake_logs(status);
CREATE INDEX idx_intake_logs_sched_id ON intake_logs(schedule_id);
CREATE INDEX idx_presc_items_presc_id ON prescription_items(prescription_id);
```

Вывод: На малых объемах данных (60 записей) прирост времени незаметен. Однако при масштабировании до миллионов записей замена Seq Scan (полное сканирование таблицы) на Index Scan (поиск по индексу) позволит сократить время выполнения с секунд до миллисекунд.

4. Хранимые процедуры и функции

Функция 1: get_daily_pill_count(p_id UUID)

Описание: Принимает UUID пациента и возвращает целое число — общее количество приемов лекарств, запланированных для этого пациента на текущую дату. Учитывает только активные назначения (текущая дата попадает в интервал `start_date` → `end_date`).

Код функции:

```
CREATE OR REPLACE FUNCTION get_daily_pill_count(p_id UUID)
RETURNS INTEGER AS $$
DECLARE
    total_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO total_count
    FROM prescriptions pr
    JOIN prescription_items pi ON pr.prescription_id = pi.prescription_id
    JOIN intake_schedules s ON pi.item_id = s.item_id
    WHERE pr.patient_id = p_id
        AND pr.start_date <= CURRENT_DATE
        AND (pr.end_date IS NULL OR pr.end_date >= CURRENT_DATE);

    RETURN total_count;
END;
$$ LANGUAGE plpgsql;
```

Тестовый вызов:

```
SELECT get_daily_pill_count('c5863bfb-0db9-491e-8a12-71ee52d36500');
```

Результат:

```
get_daily_pill_count
-----
4
(1 row)
```

![Результат функции 1](png for lab2/функция 1 .jpg)

Интерпретация: Пациент с ID `c5863bfb-...` имеет 4 запланированных приема лекарств на сегодняшний день.

Функция 2: `is_prescription_active(pr_id UUID)`

Описание: Принимает UUID назначения и возвращает `BOOLEAN` — `true` , если текущая дата попадает в интервал действия назначения, и `false` в противном случае. Используется для проверки, нужно ли пациенту продолжать прием.

Код функции:

```

CREATE OR REPLACE FUNCTION is_prescription_active(pr_id UUID)
RETURNS BOOLEAN AS $$
DECLARE
    is_active BOOLEAN;
BEGIN
    SELECT (start_date <= CURRENT_DATE AND (end_date IS NULL OR end_date >= CURRENT_DATE))
    INTO is_active
    FROM prescriptions
    WHERE prescription_id = pr_id;

    RETURN COALESCE(is_active, FALSE);
END;
$$ LANGUAGE plpgsql;

```

Тестовый вызов:

```
SELECT is_prescription_active('4fe8957b-0819-4866-831f-97793c374d75');
```

Результат:

```

is_prescription_active
-----
t
(1 row)

```

```

-- 2. Функция проверки активности назначения
CREATE OR REPLACE FUNCTION is_prescription_active(pr_id UUID)
RETURNS BOOLEAN AS $$
DECLARE
    is_active BOOLEAN;
BEGIN
    SELECT (start_date <= CURRENT_DATE AND (end_date IS NULL OR end_date >= CURRENT_DATE))
    INTO is_active
    FROM prescriptions
    WHERE prescription_id = pr_id;

    RETURN COALESCE(is_active, FALSE);
END;
$$ LANGUAGE plpgsql;

```

Интерпретация: Назначение с ID 4fe8957b-... активно (текущая дата попадает в интервал с 2025-01-01 по 2026-12-31).

Процедура 1: add_drug_to_prescription(p_presc_id UUID, p_drug_id INTEGER, p_dosage VARCHAR)

Описание: Добавляет препарат в состав назначения. Перед вставкой выполняет валидацию: проверяет, существует ли указанный препарат в справочнике `drugs_directory` . Если препарат не найден — генерируется исключение с сообщением об ошибке.

Код процедуры:

```
CREATE OR REPLACE PROCEDURE add_drug_to_prescription(
    p_presc_id UUID,
    p_drug_id INTEGER,
    p_dosage VARCHAR
)
AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM drugs_directory WHERE drug_id = p_drug_id) THEN
        RAISE EXCEPTION 'Препарат с ID % не найден в справочнике', p_drug_id;
    END IF;

    INSERT INTO prescription_items (prescription_id, drug_id, dosage)
    VALUES (p_presc_id, p_drug_id, p_dosage);
END;
$$ LANGUAGE plpgsql;
```

Тестовый вызов:

```
CALL add_drug_to_prescription('4fe8957b-0819-4866-831f-97793c374d75', 1, '15мг');
```

Результат: Query executed successfully (процедура отработала без ошибок).

```
-- 1. Процедура добавления препарата в назначение с проверкой
CREATE OR REPLACE PROCEDURE add_drug_to_prescription(
    p_presc_id UUID,
    p_drug_id INTEGER,
    p_dosage VARCHAR
)
AS $$
BEGIN
    -- Проверка существования препарата
    IF NOT EXISTS (SELECT 1 FROM drugs_directory WHERE drug_id = p_drug_id) THEN
        RAISE EXCEPTION 'Препарат с ID % не найден в справочнике', p_drug_id;
    END IF;

    INSERT INTO prescription_items (prescription_id, drug_id, dosage)
    VALUES (p_presc_id, p_drug_id, p_dosage);
END;
$$ LANGUAGE plpgsql;
```

Проверка (что препарат добавился):

```
item_id | prescription_id | drug_id | dosage
-----+-----+-----+-----
```


241 | 4fe8957b-0819-4866-831f-97793c374d75 | 1 | 15мг
(1 row)

Интерпретация: В назначение 4fe8957b-... успешно добавлен препарат с drug_id = 1 (Эналаприл) в дозировке 15мг .

Процедура 2: mark_all_as_taken(p_id UUID)

Описание: Принимает UUID пациента и автоматически создает записи в таблице intake_logs со статусом Taken для всех запланированных на сегодня приемов лекарств. Используется для быстрой массовой отметки выполнения.

Код процедуры:

```
CREATE OR REPLACE PROCEDURE mark_all_as_taken(p_id UUID)
AS $$
BEGIN
    INSERT INTO intake_logs (schedule_id, actual_time, status)
    SELECT s.schedule_id, CURRENT_TIMESTAMP, 'Taken'
    FROM prescriptions pr
    JOIN prescription_items pi ON pr.prescription_id = pi.prescription_id
    JOIN intake_schedules s ON pi.item_id = s.item_id
    WHERE pr.patient_id = p_id
        AND pr.start_date <= CURRENT_DATE
        AND (pr.end_date IS NULL OR pr.end_date >= CURRENT_DATE);
END;
$$ LANGUAGE plpgsql;
```

Тестовый вызов:

```
CALL mark_all_as_taken('c5863bfb-0db9-491e-8a12-71ee52d36500');
```

Результат: Query executed successfully (процедура отработала без ошибок).

```
-- 2. Процедура массовой отметки приемов как "Принято" для конкретного пациента
CREATE OR REPLACE PROCEDURE mark_all_as_taken(p_id UUID)
AS $$
BEGIN
    INSERT INTO intake_logs (schedule_id, actual_time, status)
    SELECT s.schedule_id, CURRENT_TIMESTAMP, 'Taken'
    FROM prescriptions pr
    JOIN prescription_items pi ON pr.prescription_id = pi.prescription_id
    JOIN intake_schedules s ON pi.item_id = s.item_id
    WHERE pr.patient_id = p_id
        AND pr.start_date <= CURRENT_DATE
        AND (pr.end_date IS NULL OR pr.end_date >= CURRENT_DATE);
END;
$$ LANGUAGE plpgsql;
```

Проверка (количество новых логов):

```
new_logs
-----
         4
(1 row)
```

Интерпретация: Для пациента c5863bfb-... было создано 4 новых записи в логах о приеме лекарств со статусом Taken .

5. Представления (VIEW)

Представление 1: v_patient_daily_schedule

Описание: Виртуальная таблица, объединяющая данные из 5 таблиц: ФИО пациента, торговое название препарата, дозировку, время приема и дни недели. Используется для главного экрана пациента — отображает только активные назначения (текущая дата попадает в интервал).

Код создания:

```
CREATE OR REPLACE VIEW v_patient_daily_schedule AS
SELECT
    p.full_name AS patient_name,
    dr.trade_name AS medication,
    pi.dosage,
    s.time_of_day,
    s.days_of_week
FROM patients p
JOIN prescriptions pr ON p.patient_id = pr.patient_id
JOIN prescription_items pi ON pr.prescription_id = pi.prescription_id
JOIN drugs_directory dr ON pi.drug_id = dr.drug_id
```

```
JOIN intake_schedules s ON pi.item_id = s.item_id
WHERE pr.start_date <= CURRENT_DATE
      AND (pr.end_date IS NULL OR pr.end_date >= CURRENT_DATE);
```

Тестовый вызов:

```
SELECT * FROM v_patient_daily_schedule LIMIT 5;
```

Результат:

```
-- 1. Расписание приемов на день для всех пациентов
-- Объединяет данные о пациенте, препарате, дозировке и времени приема.
CREATE OR REPLACE VIEW v_patient_daily_schedule AS
SELECT
    p.full_name AS patient_name,
    dr.trade_name AS medication,
    pi.dosage,
    s.time_of_day,
    s.days_of_week
FROM patients p
JOIN prescriptions pr ON p.patient_id = pr.patient_id
JOIN prescription_items pi ON pr.prescription_id = pi.prescription_id
JOIN drugs_directory dr ON pi.drug_id = dr.drug_id
JOIN intake_schedules s ON pi.item_id = s.item_id
WHERE pr.start_date <= CURRENT_DATE
      AND (pr.end_date IS NULL OR pr.end_date >= CURRENT_DATE);
```

Интерпретация: Пациент Максим Кузнецов имеет 4 запланированных приема на сегодня: Метформин (10мг в 05:03) и Лозартан (20мг в разное время).

Представление 2: v_doctor_workload

Описание: Аналитическое представление, показывающее загруженность каждого врача: количество уникальных пациентов и общее число выписанных назначений. Использует `LEFT JOIN` для отображения всех врачей, включая тех, у кого пока нет назначений.

Код создания:

```
CREATE OR REPLACE VIEW v_doctor_workload AS
SELECT
    d.specialization,
    d.doctor_id,
    COUNT(DISTINCT pr.patient_id) as unique_patients,
    COUNT(pr.prescription_id) as total_prescriptions
FROM doctors d
LEFT JOIN prescriptions pr ON d.doctor_id = pr.doctor_id
GROUP BY d.doctor_id, d.specialization;
```

Тестовый вызов:

```
SELECT * FROM v_doctor_workload LIMIT 5;
```

Результат:

```
-- 2. Загруженность врачей
-- Показывает, сколько пациентов прикреплено к каждому врачу и общее число назначений.
CREATE OR REPLACE VIEW v_doctor_workload AS
SELECT
    d.specialization,
    d.doctor_id,
    COUNT(DISTINCT pr.patient_id) as unique_patients,
    COUNT(pr.prescription_id) as total_prescriptions
FROM doctors d
LEFT JOIN prescriptions pr ON d.doctor_id = pr.doctor_id
GROUP BY d.doctor_id, d.specialization;
```

Интерпретация: В выборке представлены врачи разных специальностей с информацией о нагрузке. Например, Терапевт с ID 12cb9b76-... обслуживает 4 уникальных пациентов и имеет 4 назначения.

Заключение

В ходе работы была создана база данных с репрезентативным набором данных. Проведен анализ производительности, который подтвердил эффективность использования индексов для оптимизации JOIN-операций и фильтрации. Реализована серверная логика на языке PL/pgSQL (2 функции и 2 процедуры), а также 2 представления, объединяющих данные из нескольких таблиц. Все объекты протестированы на реальных данных из БД и показали корректные результаты.